

Foundations of Programming Languages

CG

May 1, 2026

1 Abstract syntax trees

An *abstract syntax tree* (AST) is a tree representation of the hierarchical structure of a program. Unlike a concrete parse tree, an AST omits purely syntactic detail such as parenthesis, semicolons used as separators, keywords whose meaning is already captured by the node type; retaining only the information needed to define the program's meaning.

Each *internal node* of the AST corresponds to a language construct (sequence, assignment, conditional, etc.) and each *leaf* corresponds to an atomic value or reference.

For an imperative language with mutable references, the relevant constructs are:

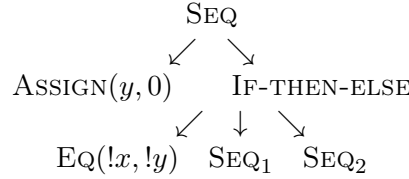
- $C_1 ; C_2$, sequential composition
- $l := e$, assignment to location l
- **if** B **then** C_1 **else** C_2 , conditional command
- **while** B **do** C , iteration
- $!l$, dereferencing a location (read the value stored at l)
- n, b , integer and boolean literals
- $e_1 \text{ op } e_2$, arithmetic or boolean combination

1.1 solved example

Build the AST of the following program:

$y := 0 ; \text{ if } !x = !y \text{ then } x := !x + 1 ; y := !y - 1 \text{ else } x := !x - 1 ; y := !y + 1$

The top level operator is the semicolon that separates the two main components, so the root is a SEQ node. Its left child is the assignment $y := 0$, and its right child is the conditional. The conditional has three children: the guard $!x = !y$, the *then*-branch $x := !x + 1 ; y := !y - 1$, and the *else*-branch (symmetric).



note that $!x$ is represented as a Deref node applied to the location x , which is distinct from x as an identifier: the location names a cell in the store, and dereferencing reads its content.

1.2 Exercises

- Build the AST of: $x := !x \times 2 ; \text{if } !x > 10 \text{ then skip else } x := !x + 1$
- Build the AST of: **while** $!i < !n$ **do** $s := !s + !i ; i := !i + 1$
- Reconstruct the source program corresponding to the following AST:
root is SEQ with left child ASSIGN($x, 5$) and right child IF-THEN-ELSE(EQ($!x, 5$), ASSIGN($y, 1$), ASS

2 Type inference

Type inference is the process by which a compiler deduces the type of an expression without explicit annotations from the programmer. the classical algorithm for the simply typed lambda calculus (and its extensions) is *Hindley Milner* (HM), which works by:

1. Assigning a fresh type variable α_i to each sub-expression whose type is unknown.
2. Generating *constraints* of the form $\tau_1 = \tau_2$ from the typing rules.
3. Solving the constraints by *unification*.

if unification fails, the program contains a type error; if it succeeds, the resulting substitution gives the most general type.

2.1 Typing rules for references

$\frac{\text{T-DEREF} \quad \Gamma \vdash !l : T}{\Gamma \vdash l : \text{ref } T}$	$\frac{\text{T-ASSIGN} \quad \Gamma \vdash l := e : \text{unit}}{\Gamma \vdash l : \text{ref } T \quad \Gamma \vdash e : T}$	$\frac{\text{T-WHILE} \quad \Gamma \vdash \text{while } B \text{ do } C : \text{unit}}{\Gamma \vdash B : \text{bool} \quad \Gamma \vdash C : \text{unit}}$
---	--	--

2.2 solved example

$\lambda f. f f$ does not typecheck

Consider `proc(f) zero?((f f))`. We try to infer the type of f .

- Let $f : \alpha$.
- The expression $f f$ requires f to be a function type, so $\alpha = \alpha \rightarrow \beta$ for some fresh β .
- But $\alpha = \alpha \rightarrow \beta$ has no finite solution: expanding repeatedly yields $\alpha = (\alpha \rightarrow \beta) \rightarrow \beta = ((\alpha \rightarrow \beta) \rightarrow \beta) \rightarrow \beta = \dots$

unification fails (occurs check). Therefore the expression has no type in the simply typed lambda calculus without recursive types.

2.3 Scala generics

```
case class MyPair[A,B](x:A,y:B)
val p = MyPair(1, "Scala")
def id[T](x:T)=x
val q = id(1)
```

the definition of `id` introduces a **polymorphic** type: it can be applied to any type, and each call site independently instantiates a T .

```
B = !n < 5
B1 = !term <= 2
C1 = res := !n
C2 = c:=!a+!b; res:=!n; !c; a:=!b; b:=!c
C = if B1 then C1 else C2; n:=!n+1
a:=1; b:=1; while B do C
```

reading off types from the operations performed:

- $n, res, a, b, c : \text{ref int}$ (assigned integers and dereferenced in arithmetic).
- $term : \text{ref int}$ (compared with 2).

- $B, B_1 : \mathbf{bool}$.
- $C_1, C_2, C : \mathbf{unit}$ (commands produce no value).
- The whole program : **unit**.

2.4 Exercises

Infer the type of the S combinator: $\lambda x. \lambda y. \lambda z. x z (y z)$. What constraints does unification generate?

Consider $\lambda f. \lambda x. f (f x)$. Does it typecheck? If so, give its most general type.

What types are inferred for **f**, **result**, and **pairs** in the following Scala snippet?

```
def f[A](lst: List[A]) = lst.head
val result = f(List(1,2,3))
val pairs  = List((1,"a"), (2,"b"))
```

Given $x : \mathbf{ref\ int}$ and $b : \mathbf{ref\ bool}$, does the following expression type-check? If so, give its type.

$b := !x = 0 ; \mathbf{if\ !b\ then\ } x := 1 \mathbf{\ else\ skip}$

3 Operational semantics

3.1 Big-step semantics

In *big-step* (natural semantics, a judgement) $\langle e, \sigma \rangle \Downarrow v$ asserts that expression e in store σ evaluates *directly* to value v , without exposing intermediate steps. Derivations are trees whose leaves are axioms and whose internal nodes are inference rules.

3.1.1 Lists: concat and append

We assume the constructors **nil** (empty list) and **cons**(h, t) (list with head h and tail t).

concat(e, ℓ), **insert element** e **at the end of** ℓ .

$$\frac{\text{CONCAT-NIL} \quad \langle \mathbf{concat}(e, \mathbf{nil}), \sigma \rangle \Downarrow \mathbf{cons}(e, \mathbf{nil}) \quad \text{CONCAT-CONS} \quad \langle \mathbf{concat}(e, \mathbf{cons}(h, t)), \sigma \rangle \Downarrow \mathbf{cons}(h, t')}{\langle \mathbf{concat}(e, t), \sigma \rangle \Downarrow t'}$$

$\text{append}(\ell_1, \ell_2)$ concatenate two lists.

$$\frac{\text{APPEND-NIL} \quad \langle \mathbf{append}(\mathbf{nil}, \ell_2), \sigma \rangle \Downarrow \ell_2 \quad \text{APPEND-CONS} \quad \frac{\langle \mathbf{append}(\mathbf{cons}(h, t_1), \ell_2), \sigma \rangle \Downarrow \mathbf{cons}(h, \ell')}{\langle \mathbf{append}(t_1, \ell_2), \sigma \rangle \Downarrow \ell'}}{\langle \mathbf{append}(\ell_1, \ell_2), \sigma \rangle \Downarrow \ell'}$$

Derivation for append We verify $\langle \mathbf{append}(\mathbf{cons}(1, \mathbf{nil}), \mathbf{cons}(2, \mathbf{nil})), \sigma \rangle \Downarrow \mathbf{cons}(1, \mathbf{cons}(2, \mathbf{nil}))$:

$$\frac{\text{APPEND-CONS} \quad \langle \mathbf{append}(\mathbf{cons}(1, \mathbf{nil}), \mathbf{cons}(2, \mathbf{nil})), \sigma \rangle \Downarrow \mathbf{cons}(1, \mathbf{cons}(2, \mathbf{nil}))}{\text{APPEND-NIL} \quad \langle \mathbf{append}(\mathbf{nil}, \mathbf{cons}(2, \mathbf{nil})), \sigma \rangle \Downarrow \mathbf{cons}(2, \mathbf{nil})}$$

3.2 Small-step semantics

In *small-step* semantics, a judgement $\langle P, \sigma \rangle \rightarrow \langle P', \sigma' \rangle$ performs a single atomic reduction. A program is fully evaluated when no rule applies (it is a value or a stuck state) or when it reduces to **skip** (for commands).

3.2.1 For-loops in SIMP

The grammar of SIMP extended with **for** is:

$$C ::= l := E \mid C ; C \mid \mathbf{if} \ B \ \mathbf{then} \ C \ \mathbf{else} \ C \mid \mathbf{while} \ B \ \mathbf{do} \ C \mid \mathbf{skip} \mid \mathbf{for} \ (C; B; C) \ \{C\}$$

The cleanest approach is to treat **for** as *syntactic sugar*:

$$\mathbf{for} \ (C_i; B; C_s) \ \{C_b\} \triangleq C_i; \mathbf{while} \ B \ \mathbf{do} \ C_b; C_s$$

If the language requires explicit small-step rules (no sugar), we give:

$$\frac{\text{FOR-INIT} \quad \langle \mathbf{for} \ (C_i; B; C_s) \ \{C_b\}, \sigma \rangle \rightarrow \langle \mathbf{for} \ (C'_i; B; C_s) \ \{C_b\}, \sigma' \rangle}{\langle C_i, \sigma \rangle \rightarrow \langle C'_i, \sigma' \rangle}$$

FOR-SKIP

$$\langle \mathbf{for} \ (\mathbf{skip}; B; C_s) \ \{C_b\}, \sigma \rangle \rightarrow \langle \mathbf{if} \ B \ \mathbf{then} \ (C_b; C_s; \mathbf{for} \ (\mathbf{skip}; B; C_s) \ \{C_b\}) \ \mathbf{else} \ \mathbf{skip}, \sigma \rangle$$

Rule FOR-INIT reduces the initialiser one step at a time. Rule FOR-SKIP fires once the initialiser has been fully executed: it checks the guard and either runs the body followed by the step expression (and loops), or terminates with **skip**.

3.3 Exercises

- Write big-step rules for **reverse**(ℓ), which reverses a list.
- Use **concat** as a helper.

Prove with a full derivation tree:

$$\langle \text{concat}(3, \text{cons}(1, \text{cons}(2, \text{nil}))), \sigma \rangle \Downarrow \text{cons}(1, \text{cons}(2, \text{cons}(3, \text{nil})))$$

- Write big-step rules for **length**(ℓ), which returns the number of elements, and evaluate $\langle \text{length}(\text{cons}(1, \text{cons}(2, \text{nil}))), \sigma \rangle \Downarrow 2$ with a derivation tree.

4 Lambda calculus

Lambda calculus is a formal system that serves as the theoretical foundation for functional programming languages. The core idea is surprisingly simple: everything reduces to functions, how they are defined and how they are applied.

4.1 basic syntax

There are only three constructs in the language:

- **Variables:** $x, y, z \dots$
- **Abstraction:** $\lambda x. e$ — defines a function with parameter x and body e .
- **Application:** $e_1 e_2$ — applies function e_1 to argument e_2 .

from just these three pieces, any computation can be expressed.

4.2 Free and Bound Variables

In $\lambda x. M$, we say x is **bound**: the abstraction defines its scope. A variable that appears outside the scope of any λ is called **free**.

α -equivalence tells us that the name of a bound variable does not matter, as long as renaming causes no conflicts:

$$\lambda x. x \equiv_{\alpha} \lambda y. y$$

4.3 Reduction Rules

Computation happens through two main rules:

- **β -reduction** — substitutes the argument into the function body:

$$(\lambda x. e) v \rightarrow e[x := v]$$

This is the fundamental computational step.

- **η -reduction** — two functions are equal if they produce the same result for every argument:

$$\lambda x. (f x) \rightarrow f \quad (\text{if } x \text{ is not free in } f)$$

4.4 Evaluation Strategies

In what order do we reduce? There are several options, each with different consequences:

1. **Normal Order:** always reduces the leftmost, outermost redex first. Guaranteed to find a normal form if one exists.
2. **Call-by-Value:** evaluates the argument before passing it to the function (used in C, Java, Python).
3. **Call-by-Name:** passes the argument unevaluated — the basis for lazy evaluation.

4.5 Church Encodings

Since lambda calculus has no built-in types or literals, we need to *encode* familiar values as functions:

- **Booleans:**

$$TRUE \equiv \lambda t. \lambda f. t$$

$$FALSE \equiv \lambda t. \lambda f. f$$

$$AND \equiv \lambda p. \lambda q. p q p$$

- **Church Numerals** — the number n applies f exactly n times:

$$0 \equiv \lambda f. \lambda x. x$$

$$1 \equiv \lambda f. \lambda x. f x$$

$$2 \equiv \lambda f. \lambda x. f (f x)$$

$$SUCC \equiv \lambda n. \lambda f. \lambda x. f (n f x)$$

4.6 Recursion: The Y Combinator

Since functions are anonymous, recursion is achieved via fixed points. Curry's Y Combinator is defined as:

$$Y = \lambda f. (\lambda x. f(x x)) (\lambda x. f(x x))$$

It satisfies the property: $Y g = g(Y g)$, enabling infinite recursion.

4.7 Recursion: The Y Combinator

Lambda functions are anonymous, so recursion is not straightforward. The solution is Curry's **fixed-point combinator**:

$$Y = \lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))$$

It satisfies $Y g = g(Y g)$, which allows recursive functions to be defined without ever naming them.

5 Type safety

Type safety is the formal guarantee that well-typed programs do not (go wrong), they never reach a state that is undefined or meaningless. It is established through two theorems that work together: **progress** and **preservation**.

5.1 progress

A well-typed expression is never *stuck*, it is either already a value, or it can take a step.

Teorema 5.1 (progress). *If $\vdash e : \tau$ and $e \rightarrow e'$, then $\vdash e' : \tau$.*

Intuitively, this rules out situations like applying an integer as if it were a function, a well-typed program always has somewhere to go.

5.2 preservation

Taking a step does not change the type of an expression.

Teorema 5.2 (preservation). *If $\vdash e : \tau$ and $e \rightarrow e'$, then $\vdash e' : \tau$*

Together, progress and preservation give us the **type safety** property: evaluation of a well-typed expression either produces a value of the expected type, or diverges; but it never crashes.

5.3 Supporting lemmas

The proofs of the both theorems rely on two key lemmas.

Lema 5.3 (Canonicity). *If $\vdash v : \tau$ and v is a value, then the shape of v is determined by τ . For example:*

- *If $\tau = \text{Bool}$, then v is either true or false.*
- *If $\tau = \tau_1 \rightarrow \tau_2$, then $v = \lambda x. e$ for some x and e .*

This lemma is what lets us case-split on the shape of a value during a proof, we know exactly what forms are possible.

Lema 5.4 (Substitution). *If $\Gamma, x : \tau' \vdash e : \tau$ and $\vdash v : \tau'$, then $\Gamma \vdash e[x := v] : \tau$.*

in plain terms, substituting a well-typed value for a variable preserves typing. This is the key step in proving Preservation for β reduction.

5.4 proof strategy: structural induction

Both theorems are proved by **structural induction** on the typing derivation. The general scheme is:

1. **Base cases:** handle values and variables directly, these are straightforward since no reduction is possible.
2. **Inductive cases:** for each typing rule (application, abstraction, conditionals, etc.), assume the result holds for the subexpressions (*induction hypothesis*), then show it holds for the whole expression.
3. **Use the lemmas:** Canonicity tells you what shape a value has; Substitution handles the β -reduction case in Preservation.

the structure of the proof closely mirrors the structure of the language, one case per typing rule. Once you see the pattern it becomes mechanical.

6 Denotational semantics

While operational semantics describes *how* a program executes step by step, denotational semantics asks a different question: *what does a program mean?*. the idea is to assign each program a mathematical object, its denotation, in a way that is compositional; the meaning of compound expression is built entirely from the meanings of its parts.

6.1 interpretation functions

The central tool is the **interpretation function** $\llbracket \cdot \rrbracket$, which maps syntactic expressions to mathematical values. The double brackets are just notation, is like a translation from syntax into math.

6.2 the state domain

Programs do not compute in a vacuum, they read and write variables. We model this with a *state*, a snapshot of memory at any point during execution:

$$\Sigma = Var \rightarrow Val$$

A state $\sigma \in \Sigma$ assigns a value to every variable. With this, we can give meaning to both expressions and commands.

6.3 meaning of expressions

An arithmetic or boolean expression takes a state and returns a value:

$$\llbracket e \rrbracket : \Sigma \rightarrow Val$$

For example, the meaning of $x + 1$ in state σ is just $\sigma(x) + 1$. There is no mutation here, expressions are pure functions over states.

6.4 meaning fo commands

Commands are more interesting: they *transform* states. A command takes an initial state and produces a final one:

$$\llbracket C \rrbracket : \Sigma \rightarrow \Sigma_{\perp}$$

The $\perp$ subscript is important. It accounts for *non-termination*, a **while** loop may never finish, so the result is not always a state. $\Sigma_{\perp} = \Sigma \cup \{\perp\}$ adds a special element $\perp$ (“bottom”) to represent divergence.

6.5 fixed points and iteration

The **while** loop is the tricky case. We cannot define its meaning directly, since it may run for an unbounded number of steps. instead, we use **Kleene’s fixed-point theorem**.

The idea is to build the meaning of **while** b **do** C as the least fixed point of a functional Φ :

$$\Phi(f)(\sigma) = \begin{cases} f(\llbracket C \rrbracket(\sigma)) & \text{if } \llbracket b \rrbracket(\sigma) = \text{true} \\ \sigma & \text{if } \llbracket b \rrbracket(\sigma) = \text{false} \end{cases}$$

We approximate the meaning from below: $\Phi^0 = \perp$, $\Phi^1 = \Phi(\perp)$, $\Phi^2 = \Phi(\Phi(\perp))$, and so on. The least fixed point $\text{lfp}(\Phi) = \bigsqcup_{n \geq 0} \Phi^n$ captures *all finite unrollings* of the loop, which is exactly what the loop computes.

6.6 equivalence with operational semantics

A natural question arises: do both styles of semantics agree? The answer is yes.

Teorema 6.1 (Equivalence). *For all commands C , initial states σ , and final states σ' :*

$$\langle C, \sigma \rangle \Downarrow \sigma' \iff \llbracket C \rrbracket(\sigma) = \sigma'$$

operational semantics tells us *how* the program runs, and denotational semantics tells us *what* it computes, and they agree on every input/output pair. Neither one is more “correct”; they are two lenses on the same truth.

7 Axiomatic semantics (Hoare logic)